Design Facebook News Feed

SystemsExpert design walkthrough

Clarifying questions

- Ranked or chronological?
- Text only or media too?
- Friends only, or pages/groups?
- How fresh must the feed be?

Functional requirements

- Create posts
- See a feed of friends' posts
- Like/comment counts on posts

Non-functional requirements

- Feed load p95 < 500 ms
- Eventual consistency acceptable (seconds)
- Billions of users, highly available

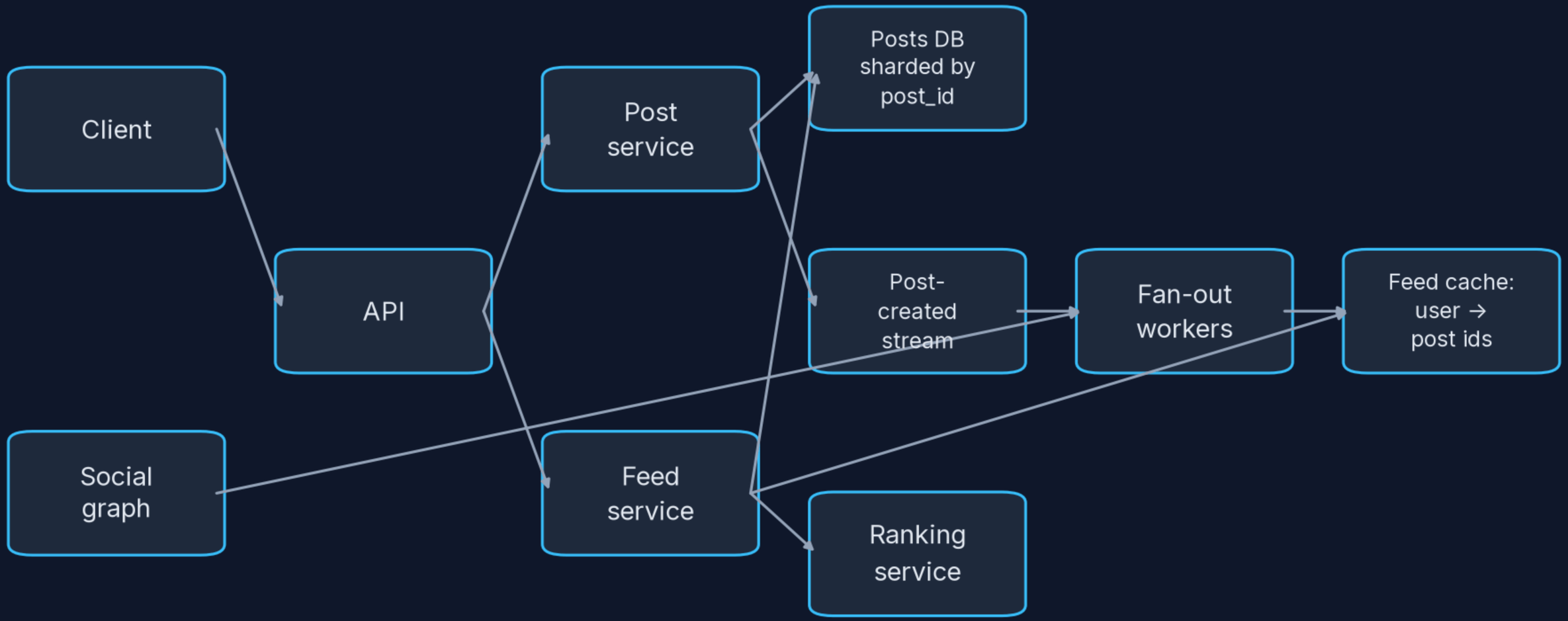
Capacity estimates

- ~2B users, ~1B DAU, each opens feed ~10x/day → ~100k feed reads/s avg, ~300k peak
- ~500M posts/day → ~6k writes/s
- Average ~500 friends; celebrities millions of followers

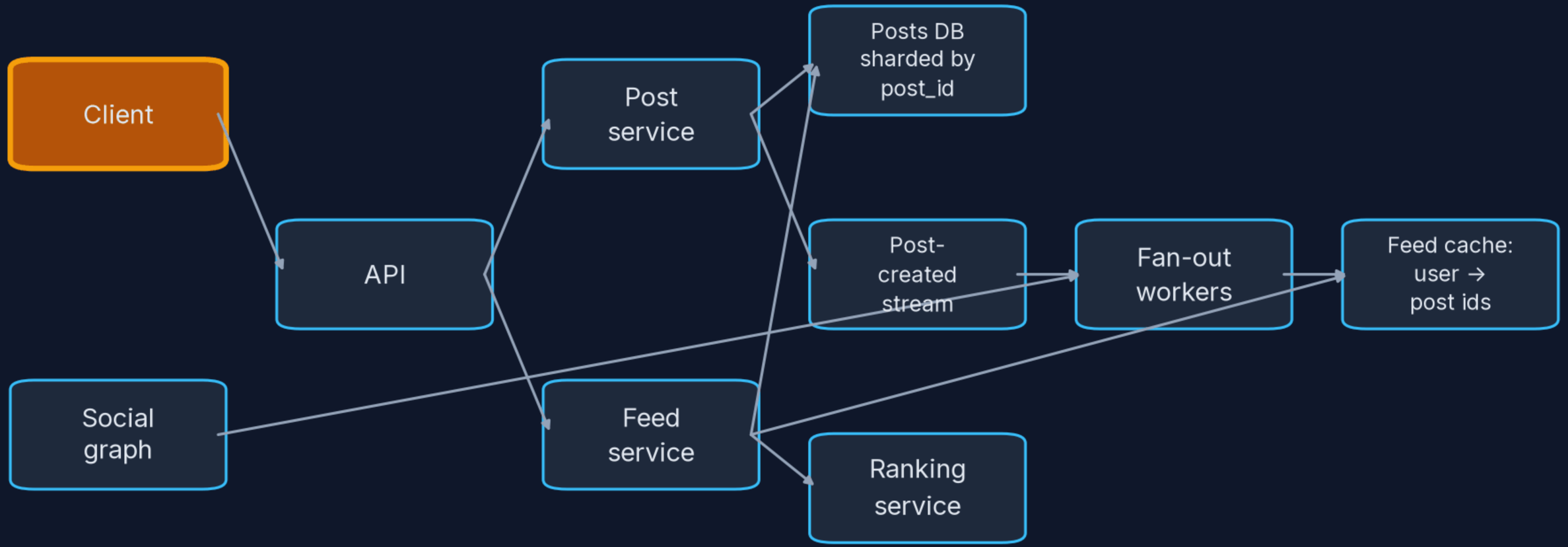
Capacity: worked out

- 1B DAU × 10 opens/day = 10B reads/day
- 10B ÷ 86,400 ≈ 115k reads/s; peak ~3× ≈ 350k/s
- 500M posts/day ≈ 6k writes/s
- Fan-out: 6k/s × 500 friends ≈ 3M cache writes/s

High-level architecture

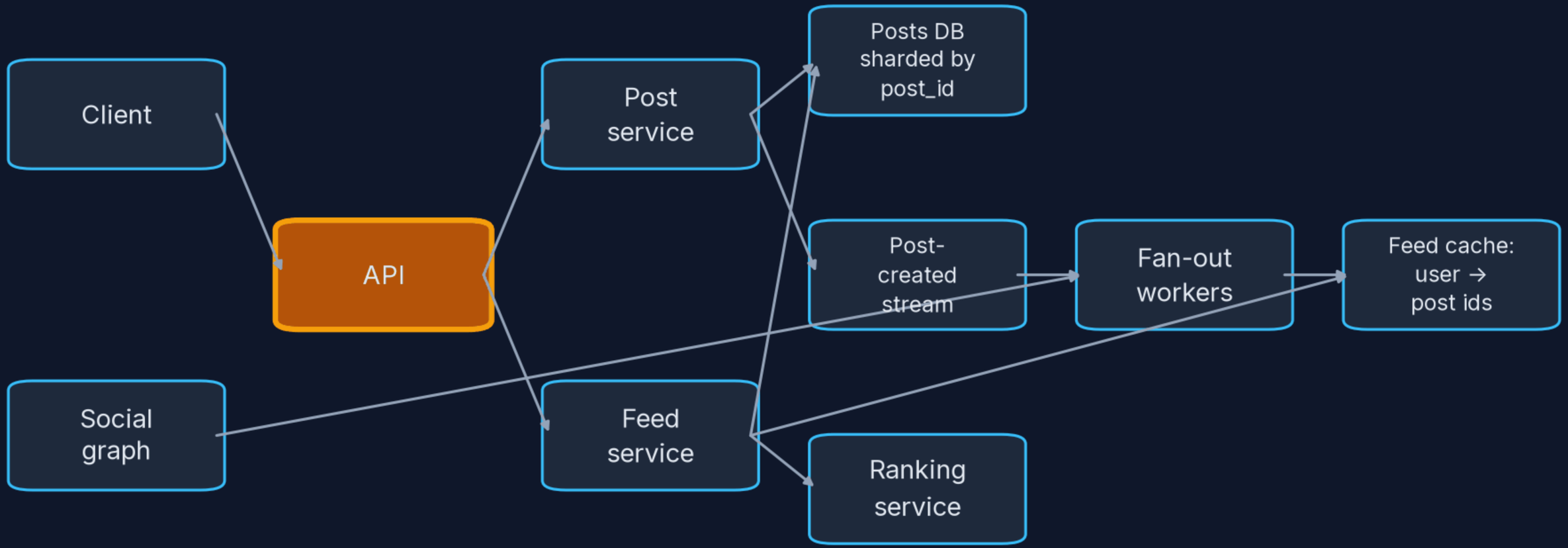


Request flow: Open the feed



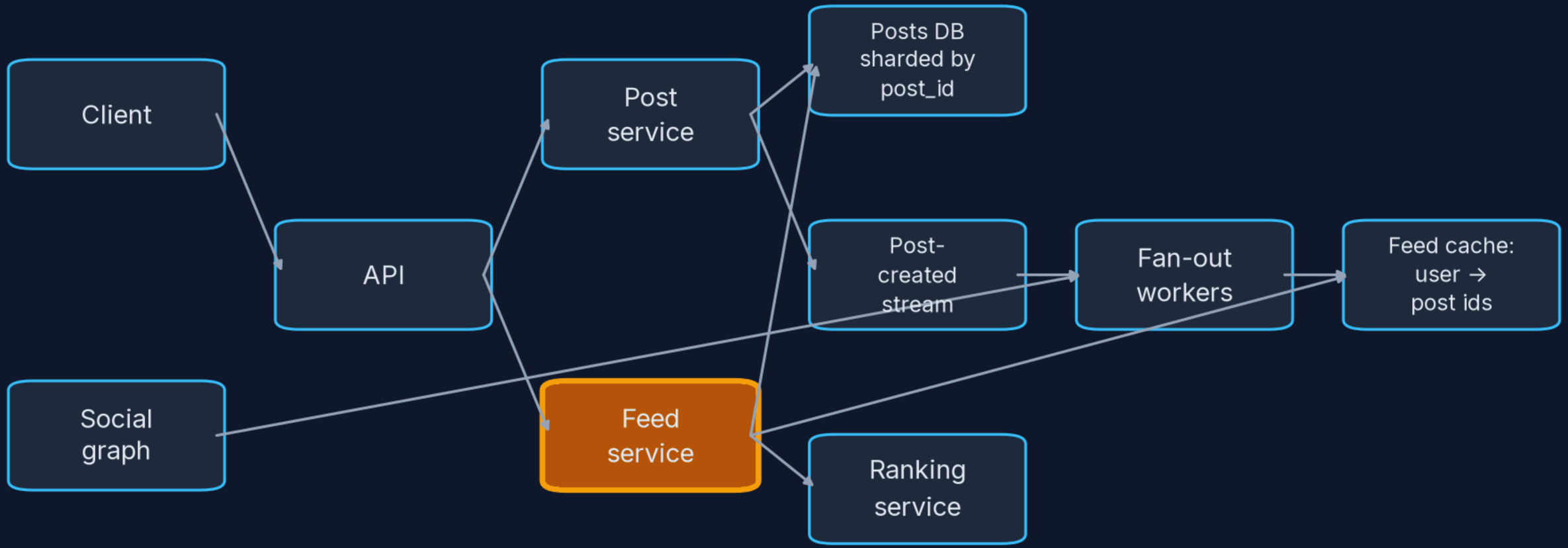
1. User opens the app

Request flow: Open the feed



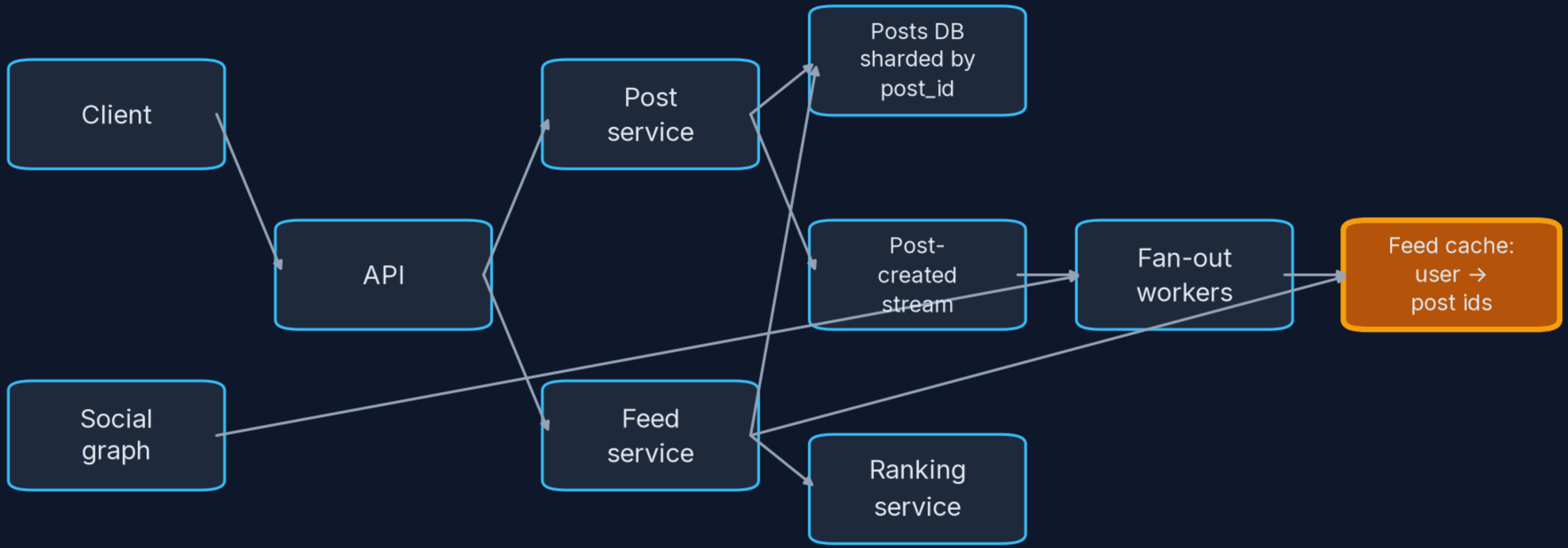
2. API routes to feed service

Request flow: Open the feed



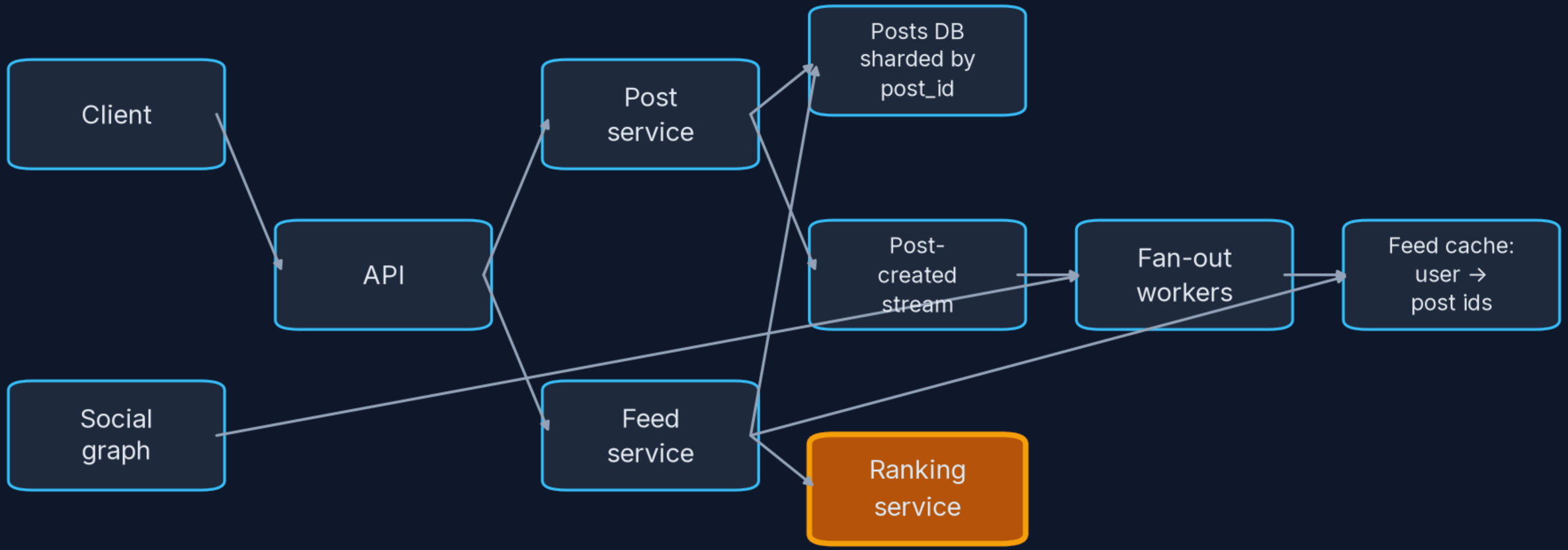
3. Read precomputed post IDs

Request flow: Open the feed



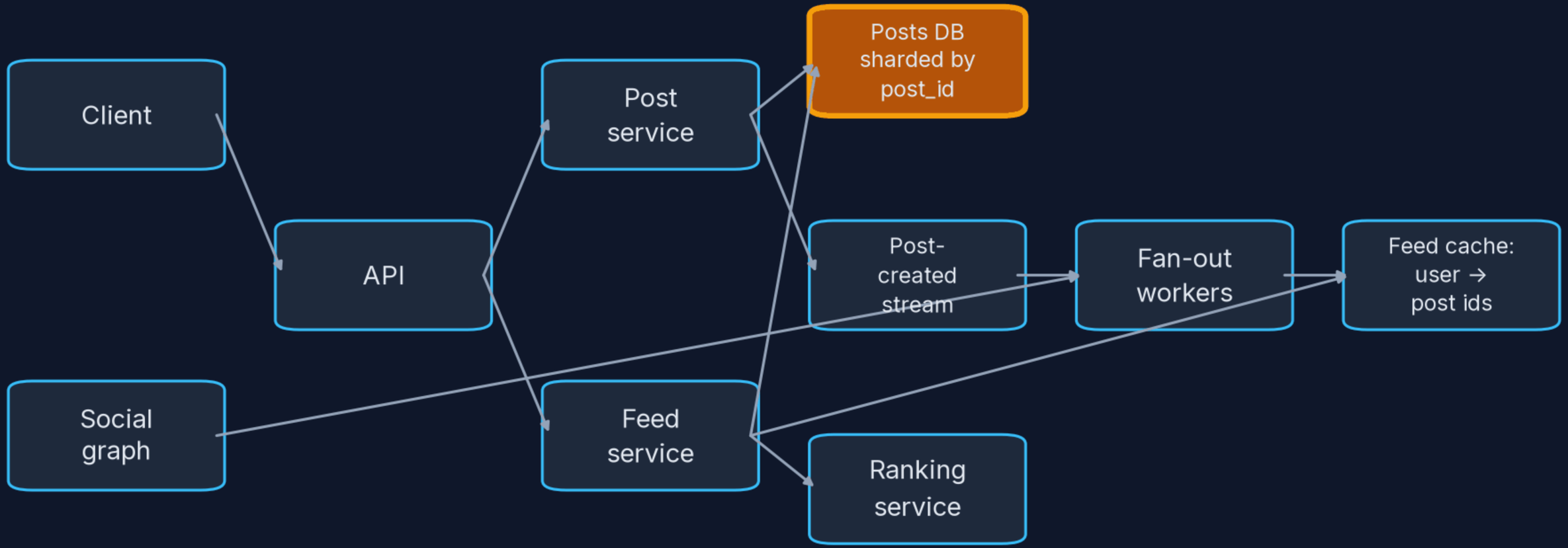
4. Cache returns candidates

Request flow: Open the feed



5. Rank candidates

Request flow: Open the feed



6. Hydrate the top posts

Data model

- Posts(id, author_id, body, media_urls, created_at)
- Edges(user_id, friend_id)
- Feed_cache: user_id → list of (post_id, score) capped ~500

API

```
POST /posts {body, media}
GET  /feed?cursor=...&limit=20 -> posts[], nextCursor
```

Fan-out on write vs read

- Push post ids into followers' precomputed feeds for normal users (fast reads).
- For celebrities (> ~10k followers) skip fan-out and merge their recent posts at read time - hybrid model.

Ranking

- Feed service fetches candidate ids, hydrates posts in batch, scores with a model (affinity, recency, engagement) and paginates with a stable cursor.

Storage

- Posts in a sharded KV/SQL store; media in blob store + CDN; feed cache in Redis/Memcache clusters sharded by user_id, rebuildable from source.

Key decisions

- Fan-out on write for most users → fast reads
- Fan-out on read for celebrities → avoids millions of writes per post
- Feed cache stores IDs only → small, cheap to update
- Separate ranking service → iterate on models independently

Trade-offs & alternatives

- Precomputed feeds use a lot of memory but give low latency
- Hybrid fan-out adds read-time merge complexity
- Eventual consistency means your own post may appear slightly late - write-through to your own feed fixes it

Recap

- Clarify scope, then functional + non-functional needs
- Estimate load to size storage and fan-out
- Core pieces: Client, API, Post service, Posts DB sharded by post_id, Post-created stream
- Know your trade-offs and failure modes